# AI Assisted Coding

# Assignment 6.1

P Sree Varshini

2303A52336

Batch-40

## Experiment 6: AI-Based Code Completion: Working with suggestions for classes, loops, conditionals

**Task Description #1** (AI-Based Code Completion for Loops)

Task: Use an AI code completion tool to generate a loop-based program.

Prompt:

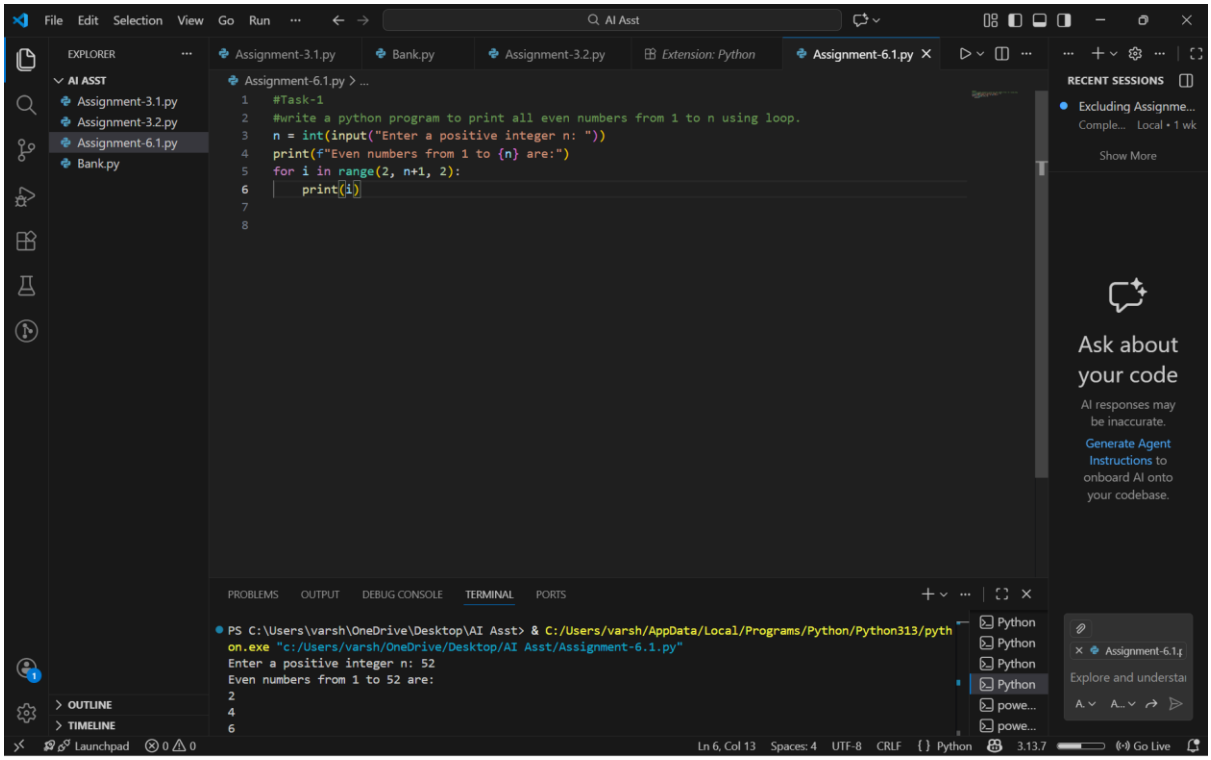
“Generate Python code to print all even numbers between 1 and N using a loop.”

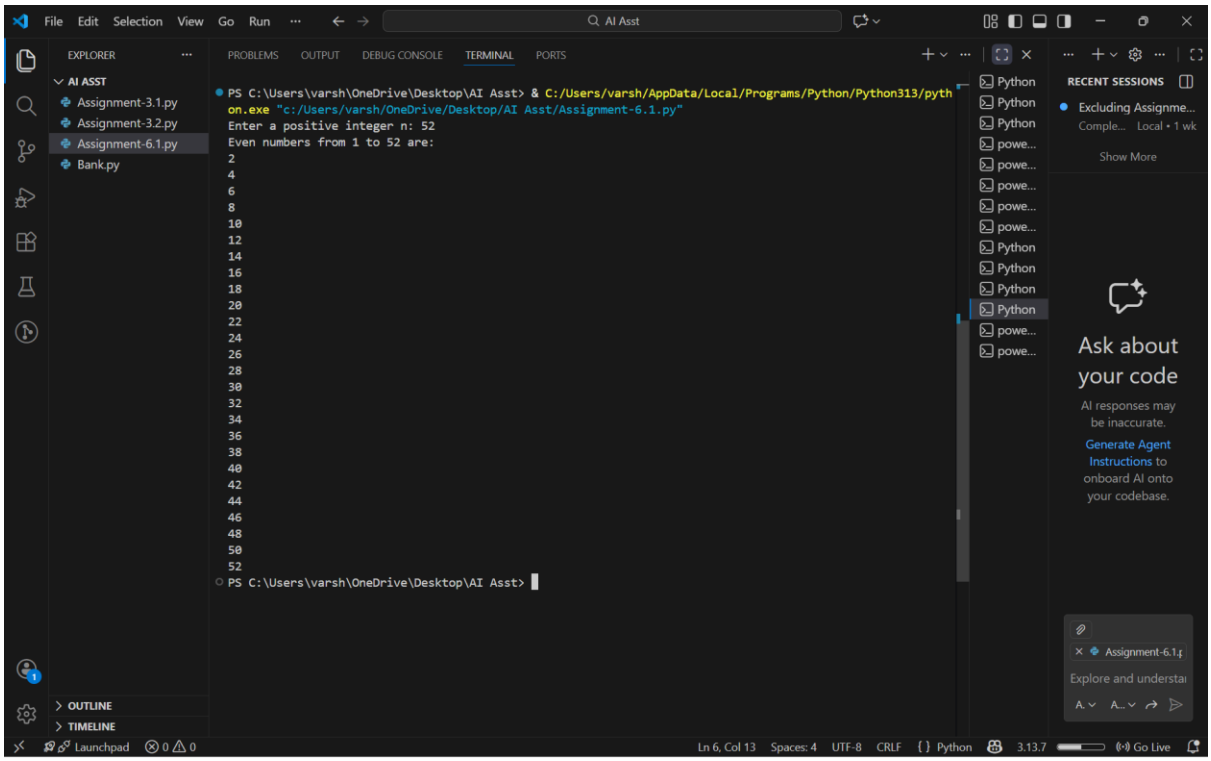
Expected Output:

• AI-generated loop logic.

• Identification of loop type used (for or while).

• Validation with sample inputs.




**Analysis:**

The program asks the user to enter a number **n**.

The number is stored in the variable **n**.

The loop starts from **2** because 2 is the first even number.

The loop increases by **2** each time.

Only even numbers are printed up to **n**.

**Task Description #2** (AI-Based Code Completion for Loop with Conditionals)

Task: Use an AI code completion tool to combine loops and conditionals.

Prompt:

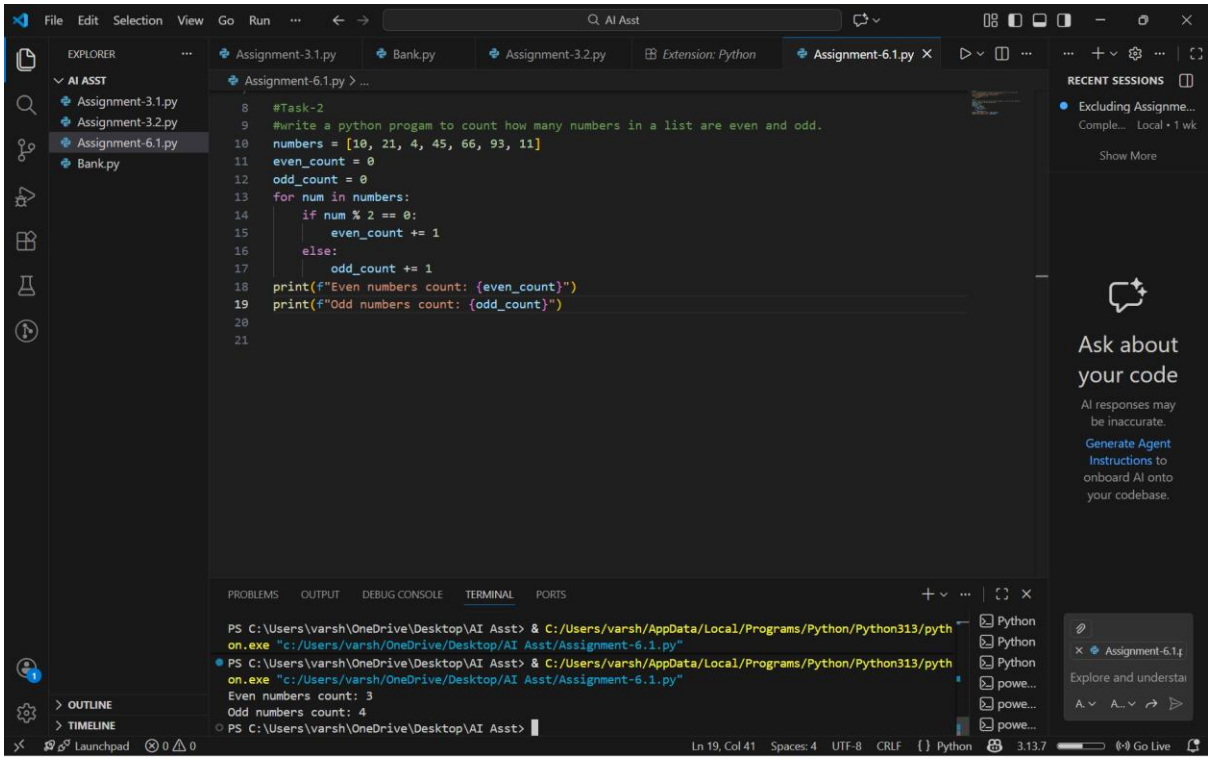
“Generate Python code to count how many numbers in a list are even and odd.”

Expected Output:

• AI-generated code using loop and if condition.

• Correct count validation.

• Explanation of logic flow.



**Analysis:**

A list of numbers is stored in the variable **numbers**.

Two variables, **even count** and **odd count**, are set to 0.

The program checks each number in the list using a **for loop**.

If a number is divisible by 2, it is **even**, so even count is increased.

Otherwise, the number is **odd**, so odd count is increased.

Finally, the program prints the total count of even and odd numbers.

**Task Description #3** (AI-Based Code Completion for Class Attributes Validation)

Task: Use an AI tool to complete a Python class that validates user input.

Prompt:

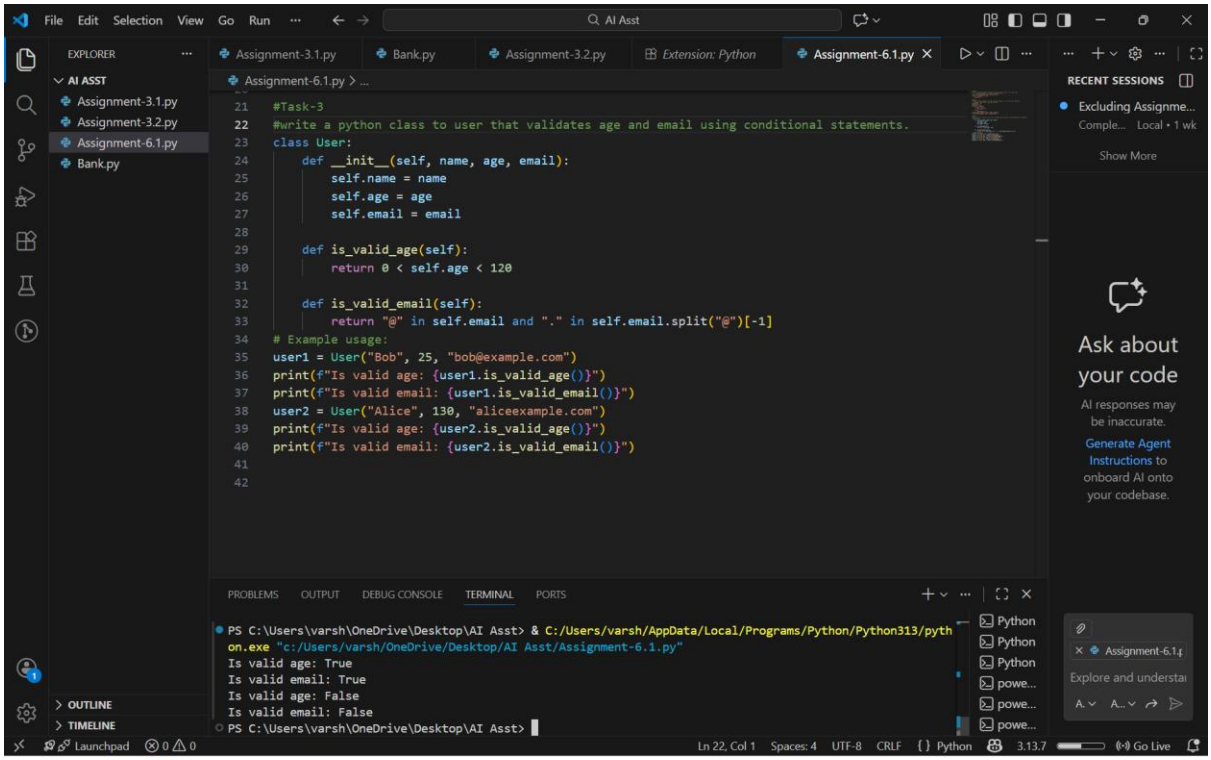
“Generate a Python class User that validates age and email using conditional statements.”

Expected Output:

• AI-generated class with validation logic.

• Verification of condition handling.

• Test cases for valid and invalid inputs.




**Analysis:**

A class named **User** is created.

The init method stores the **name, age, and email** of the user.

The method **is valid age()** checks if the age is between **1 and 119**.

The method **is valid email()** checks if the email:

- contains **@**
- has a **.** after @

A user object is created and the validation results are printed.

The same checks are done for another user with invalid details.

**Task Description #4** (AI-Based Code Completion for Classes)

Task: Use an AI code completion tool to generate a Python class for managing student details.

Prompt:

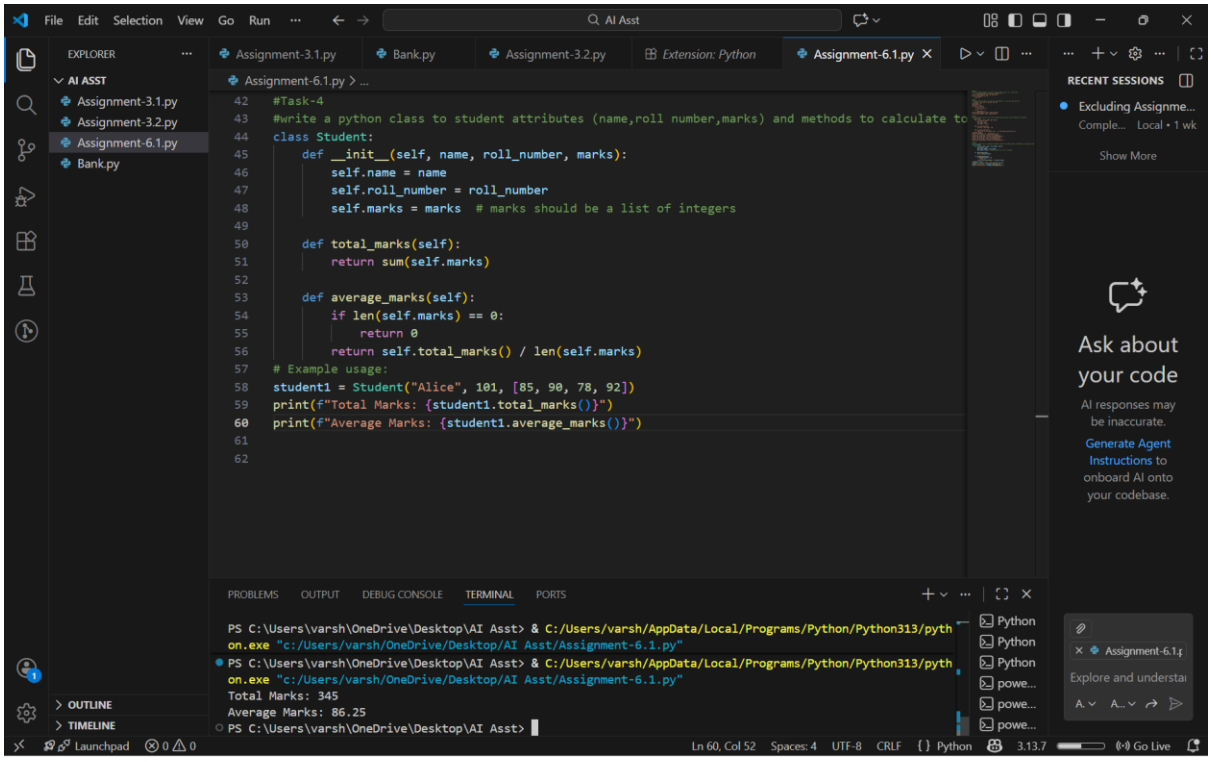
“Generate a Python class Student with attributes (name, roll number, marks) and methods to calculate total and average marks.”

Expected Output:

• AI-generated class code.

• Verification of correctness and completeness of class structure.

• Minor manual improvements (if needed) with justification.




**Analysis:**

A class named **Student** is created.

The __init__ method stores the student’s **name, roll number, and marks**.

The **total marks()** method adds all the marks and returns the total.

The **average marks()** method:

- checks if the marks list is empty
- calculates the average by dividing total marks by number of subjects

A student object is created and the total and average marks are printed.

**Task Description 5** (AI-Assisted Code Completion Review)

Task: Use an AI tool to generate a complete Python program using classes, loops, and conditionals together.

Prompt:

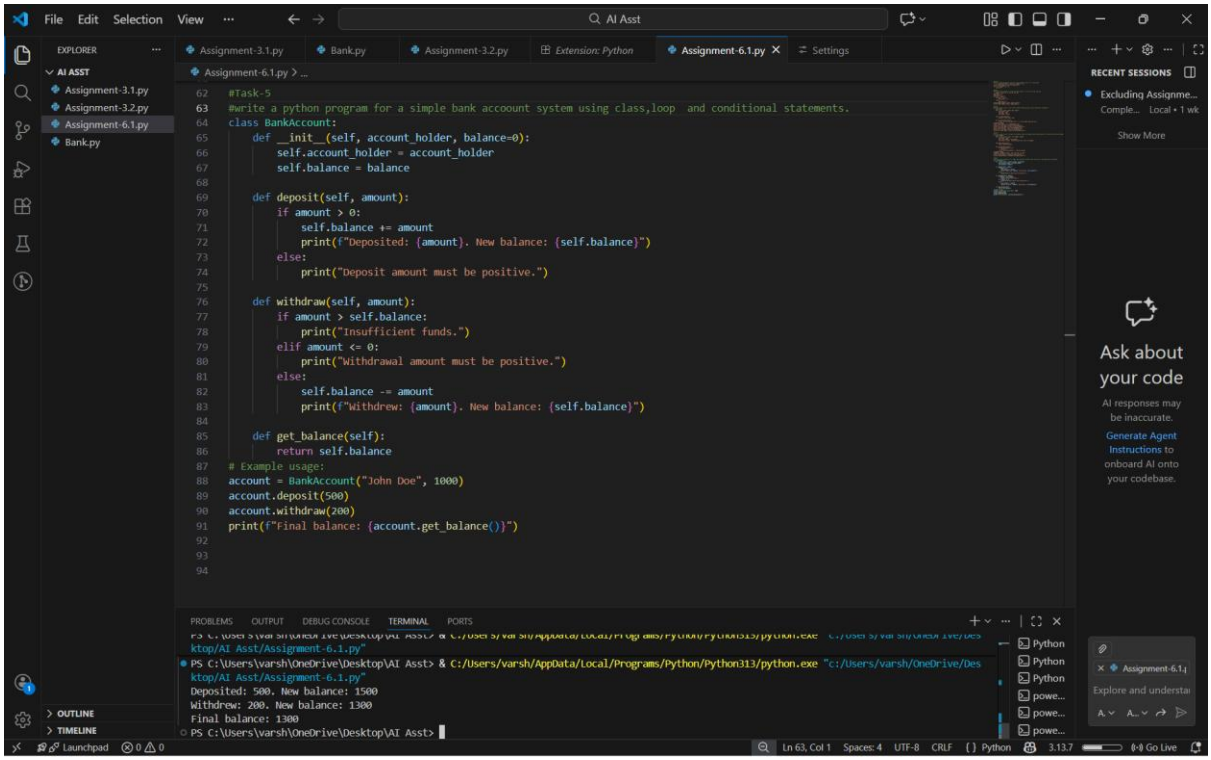
“Generate a Python program for a simple bank account system using class, loops, and conditional statements.”

Expected Output:

• Complete AI-generated program.

• Identification of strengths and limitations of AI suggestions.

• Reflection on how AI assisted coding productivity.




**Analysis:**

A class named **BankAccount** is created.

The  init method stores the **account holder name** and **balance**.

The **deposit()** method:

- checks if the amount is positive
- adds the amount to the balance

The **withdraw()** method:

- checks if there is enough balance
- checks if the amount is positive
- subtracts the amount from the balance

The **get_balance()** method returns the current balance.

The program performs deposit, withdrawal, and prints the final balance.